or you perform an operation on a user-defined type that may throw.

First up, you have the call to `distance` ❷, which performs operations on the user-supplied iterator type. Because you haven't yet done any work, and this is on the calling thread, it's fine. Next up, you have the allocation of the `results` vector ❸ and the `threads` vector ❹. Again, these are on the calling thread, and you haven't done any work or spawned any threads, so this is fine. Of course, if the construction of `threads` throws, the memory allocated for `results` will have to be cleaned up, but the destructor will take care of that for you.

Skipping over the initialization of `block_start` ❺ because that's similarly safe, you come to the operations in the thread-spawning loop ❻, ❼, ❽. Once you've been through the creation of the first thread at ❼, you're in trouble if you throw any exceptions; the destructors of your new `std::thread` objects will call `std::terminate` and abort your program. This isn't a good place to be.

The call to `accumulate_block` ❾ can potentially throw, with similar consequences; your thread objects will be destroyed and call `std::terminate`. On the other hand, the final call to `std::accumulate` ❿ can throw without causing any hardship, because all the threads have been joined by this point.

That's it for the main thread, but there's more: the calls to `accumulate_block` on the new threads might throw at ❶. There aren't any `catch` blocks, so this exception will be left unhandled and cause the library to call `std::terminate()` to abort the application.

In case it's not glaringly obvious, *this code isn't exception-safe*.

#### Adding Exception Safety

OK, so we've identified all the possible `throw` points and the nasty consequences of exceptions. What can you do about it? Let's start by addressing the issue of the exceptions thrown on your new threads.